7.1 ClusterInfo 및 Gossip 프로토콜 분석

개요

Solana의 Gossip 프로토콜은 네트워크 내 모든 노드들이 클러스터 정보를 공유하고 동기화하는 핵심 메커니즘입니다. 이 시스템은 분산 환경에서 노드 발견, 상태 전파, 그리고 네트워크 파티션 감지 및 복구를 담당합니다.

ClusterInfo 구조체 분석

핵심 구조체 정의

```
pub struct ClusterInfo {
/// The network
pub gossip: CrdsGossip,
/// set the keypair that will be used to sign crds values generated
keypair: RwLock<Arc<Keypair>>,
/// Network entrypoints
entrypoints: RwLock<Vec<LegacyContactInfo>>,
outbound_budget: DataBudget,
my_contact_info: RwLock<ContactInfo>,
ping_cache: Mutex<PingCache>,
stats: GossipStats,
socket: UdpSocket,
```

```
local_message_pending_push_queue: Mutex<Vec<CrdsValue>>,
contact_debug_interval: u64,
contact_save_interval: u64,
instance: RwLock<NodeInstance>,
contact_info_path: PathBuf,
socket_addr_space: SocketAddrSpace,
}
```

주요 컴포넌트 설명

1. **CrdsGossip**: CRDS(Conflict-free Replicated Data Store) 기반의 가십 데이터 관리
2. **ContactInfo**: 노드의 네트워크 주소 및 서비스 포트 정보
3. **PingCache**: 네트워크 지연 측정 및 연결성 확인을 위한 캐시
4. **DataBudget**: 아웃바운드 트래픽 제한 관리

Gossip 프로토콜 메시지 유형

Protocol 열거형 정의

```
#[derive(Serialize, Deserialize, Debug, AbiEnumVisitor, AbiExample)]
pub(crate) enum Protocol {
/// Pull 요청: 특정 필터 조건에 맞는 데이터 요청
PullRequest(CrdsFilter, CrdsValue),
/// Pull 응답: 요청된 데이터 전송
```

```
PullResponse(Pubkey, Vec<CrdsValue>),

/// Push 메시지: 새로운 데이터 전파

PushMessage(Pubkey, Vec<CrdsValue>),

/// Prune 메시지: 불필요한 연결 제거

PruneMessage(Pubkey, PruneData),

/// Ping 메시지: 연결성 확인

PingMessage(Ping),

/// Pong 메시지: Ping에 대한 응답

PongMessage(Pong),

}
```

메시지 처리 흐름

1. **Pull 메커니즘**

- 노드가 다른 노드에게 특정 데이터를 요청
- `CrdsFilter` 를 사용하여 필요한 데이터만 선별적으로 요청
- 네트워크 대역폭 효율성 확보

2. **Push 메커니즘**

- 새로운 정보를 네트워크에 능동적으로 전파
- 빠른 정보 전파를 위한 메커니즘
- 중복 전송 방지를 위한 필터링 적용

3. **Prune 메커니즘**

- 불필요한 연결을 제거하여 네트워크 효율성 향상
- 네트워크 토폴로지 최적화

CRDS (Conflict-free Replicated Data Store) 구조

CRDS 핵심 개념

```
pub struct Crds {
/// Stores the map of labels and values
table: IndexMap<CrdsValueLabel, VersionedCrdsValue>,
cursor: Cursor,
shards: CrdsShards,
nodes: IndexSet<usize>,
votes: BTreeMap<u64, usize>,
epoch_slots: BTreeMap<u64, usize>,
duplicate_shreds: BTreeMap<u64, usize>,
records: HashMap<Pubkey, IndexSet<usize>>,
entries: BTreeMap<u64, usize>,
purged: VecDeque<(Hash, u64)>,
shred_versions: HashMap<Pubkey, u16>,
stats: Mutex<CrdsStats>,
}
```

데이터 저장 및 관리

1. **라벨 기반 저장**: `CrdsValueLabel` 을 키로 사용하여 데이터 저장
2. **버전 관리**: `VersionedCrdsValue` 를 통한 데이터 버전 추적

3. **샤딩**: `CrdsShards` 를 통한 데이터 분산 저장

4. **인덱싱**: 다양한 데이터 타입별 인덱스 유지

GossipService 구현 분석

서비스 초기화

```
impl GossipService {

pub  fn  new(

cluster_info: &Arc<ClusterInfo>,

bank_forks: Option<Arc<RwLock<BankForks>>>,

gossip_socket: UdpSocket,

gossip_validators: Option<HashSet<Pubkey>>,

should_check_duplicate_instance: bool,

stats_reporter_sender: Option<Sender<Box<dyn FnOnce() + Send>>>,

exit: Arc<AtomicBool>,

) -> Self
```

스레드 구조

GossipService는 다음과 같은 스레드들로 구성됩니다:

1. **t_receiver**: 네트워크에서 들어오는 가십 메시지 수신

2. **t_socket_consume**: 수신된 메시지 처리 및 소비

3. **t_listen**: 가십 메시지 청취 및 응답 준비

4. **t_gossip**: 능동적인 가십 전파 및 Pull 요청

5. **t_responder**: 처리된 응답 메시지 전송

네트워크 발견 메커니즘

```
pub fn discover(
keypair: Option<Keypair>,
entrypoint: Option<&SocketAddr>,
num_nodes: Option<usize>,
timeout: Duration,
find_nodes_by_pubkey: Option<&[Pubkey]>,
find_node_by_gossip_addr: Option<&SocketAddr>,
my_gossip_addr: Option<&SocketAddr>,
my_shred_version: u16,
socket_addr_space: SocketAddrSpace,
) -> std::io::Result<(Vec<ContactInfo>, Vec<ContactInfo>)>
```

네트워크 토폴로지 및 계층 구조

계층적 네트워크 구조

Solana의 가십 네트워크는 다음과 같은 계층 구조를 가집니다:

1. **Layer 0**: 리더 노드
2. **Layer 1**: 최대한 많은 노드들을 수용 (약 2^10개)
3. **Layer 2**: 나머지 모든 노드들 (약 2^20개까지 확장 가능)

연결 관리

```
impl ClusterInfo {
fn refresh_push_active_set(
&self,
recycler: &PacketBatchRecycler,
stakes: &HashMap<Pubkey, u64>,
gossip_validators: Option<&HashSet<Pubkey>>,
sender: &PacketBatchSender,
)
```

이 함수는 활성 연결 집합을 주기적으로 갱신하여 네트워크 효율성을 유지합니다.

성능 최적화 기법

1. 데이터 필터링

```
// 스테이크 기반 필터링
fn retain_staked(values: &mut Vec<CrdsValue>, stakes: &HashMap<Pubkey, u64>) {
values.retain(|value| {
match value.data {
CrdsData::ContactInfo(_) => true,
CrdsData::Vote(_, _) => true,
CrdsData::EpochSlots(_, _) => true,
```

```
// 스테이크가 충분한 노드만 유지
_ => {
let stake = stakes.get(&value.pubkey()).copied();
stake.unwrap_or_default() >= MIN_STAKE_FOR_GOSSIP
}
}
})
}
```

2. 병렬 검증

```
impl Protocol {
fn par_verify(self, stats: &GossipStats) -> Option<Self> {
match self {
Protocol::PullResponse(from, data) => {
let size = data.len();
let data: Vec<_> = data.into_par_iter()
.filter(Signable::verify)
.collect();
// 검증 실패한 데이터 통계 업데이트
if size != data.len() {
stats.gossip_pull_response_verify_fail
.add_relaxed((size - data.len()) as u64);
```

```
}
if data.is_empty() { None } else { Some(Protocol::PullResponse(from, data)) }
}
// 다른 프로토콜 타입들도 유사하게 병렬 검증
}
}
}
```

3. 트래픽 제한

```
/// 가십 트래픽 하드 리밋 (1Gbps 순수 가십 트래픽 처리 가능)
const MAX_GOSSIP_TRAFFIC: usize = 128_000_000 / PACKET_DATA_SIZE;

/// Push 메시지 최대 페이로드 크기
const PUSH_MESSAGE_MAX_PAYLOAD_SIZE: usize = PACKET_DATA_SIZE - 44;
```

보안 및 검증

1. 디지털 서명 검증

모든 가십 메시지는 디지털 서명을 통해 검증됩니다:

```
impl Signable for PruneData {
fn signable_data(&self) -> Cow<[u8]> {
let data = SignData {
```

```
pubkey: &self.pubkey,

prunes: &self.prunes,

destination: &self.destination,

wallclock: self.wallclock,

};

Cow::Owned(serialize(&data).expect("serialize PruneData"))

}

}
```

2. 중복 인스턴스 검사

```
// 중복 인스턴스 실행 방지

should_check_duplicate_instance: bool
```

3. 소켓 주소 공간 제한

```
socket_addr_space: SocketAddrSpace
```

네트워크 보안을 위해 허용된 주소 공간만 사용합니다.

모니터링 및 통계

GossipStats 구조체

```
pub struct GossipStats {
pub gossip_pull_request_verify_fail: Counter,
pub gossip_pull_response_verify_fail: Counter,
pub gossip_push_msg_verify_fail: Counter,
pub gossip_prune_msg_verify_fail: Counter,
pub gossip_ping_msg_verify_fail: Counter,
pub gossip_pong_msg_verify_fail: Counter,
pub new_pull_requests_pings_count: Counter,
pub packets_sent_gossip_requests_count: Counter,
}
```

연결 정보 추적

```
impl ClusterInfo {
pub fn contact_info_trace(&self) -> String {
// 네트워크 내 모든 노드의 연결 상태 추적
// 샤드 버전, 스파이 노드, 다른 샤드 노드 수 계산
}
pub fn rpc_info_trace(&self) -> String {
// RPC 서비스 제공 노드들의 상태 추적
// IP 주소, 포트, 버전 정보 등 표시
}
```

```
}
```

결론

Solana의 Gossip 프로토콜은 다음과 같은 특징을 가집니다:

1. **확장성**: 계층적 네트워크 구조를 통한 대규모 네트워크 지원
2. **효율성**: Pull/Push 메커니즘과 데이터 필터링을 통한 대역폭 최적화
3. **신뢰성**: CRDS 기반의 충돌 없는 데이터 복제 및 동기화
4. **보안성**: 디지털 서명 기반의 메시지 검증
5. **모니터링**: 상세한 통계 및 추적 기능

이러한 설계를 통해 Solana는 수천 개의 검증자가 참여하는 대규모 네트워크에서도 효율적인 정보 전파와 합의를 달성할 수 있습니다.